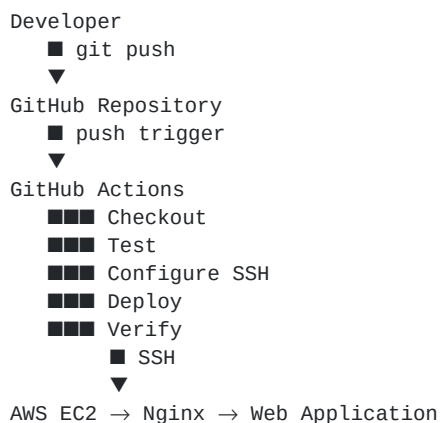


# GitHub Actions → AWS EC2

## Complete Classroom CI/CD Demo Guide

Step-by-step instructions, explanations, troubleshooting and student checklist

**Demo outcome:** a developer pushes code to GitHub; GitHub Actions checks it and automatically deploys it to Ubuntu EC2 over SSH; Nginx serves the updated site.



# 1. Learning objectives

Students should be able to:

- Explain Git vs GitHub vs GitHub Actions.
- Create a Git repository and push source code to GitHub.
- Create and understand a GitHub Actions workflow.
- Understand triggers, jobs, steps and runners.
- Use GitHub Secrets for deployment credentials.
- Deploy a web application from GitHub Actions to AWS EC2.
- Demonstrate continuous deployment by pushing a new application version.
- Troubleshoot trigger and SSH/network failures.

## Key terminology

| Term           | Meaning                                                       |
|----------------|---------------------------------------------------------------|
| Git            | Distributed version-control system.                           |
| GitHub         | Platform for hosting Git repositories and collaboration.      |
| GitHub Actions | GitHub automation and CI/CD capability.                       |
| Runner         | Machine that executes GitHub Actions jobs.                    |
| EC2            | AWS virtual machine used as deployment target.                |
| Nginx          | Web server serving the deployed files.                        |
| GitHub Secret  | Repository value kept outside source code for sensitive data. |

## 2. Prerequisites and lab design

| Component        | Choice                                                               |
|------------------|----------------------------------------------------------------------|
| Cloud            | AWS                                                                  |
| Target           | Ubuntu Server 24.04 LTS on EC2                                       |
| Instance         | t2.micro or t3.micro, subject to current account/region availability |
| Web server       | Nginx                                                                |
| Application      | Static HTML + CSS                                                    |
| Source control   | Git + GitHub                                                         |
| CI/CD            | GitHub Actions                                                       |
| Deployment       | SSH + scp                                                            |
| Target directory | /var/www/html                                                        |

### Prerequisites

- AWS account with permission to create EC2 and security groups.
- GitHub account with repository creation permission.
- Git installed locally; verify with `git --version`.
- EC2 key pair available; never expose the private key.
- EC2 public IP available during the lab.

**Teaching note:** the application is intentionally simple so students focus on CI/CD rather than application development.

### 3. Create and prepare AWS EC2

Launch an Ubuntu Server 24.04 LTS EC2 instance. Choose a small instance suitable for a classroom lab and create/select an EC2 key pair.

#### Security group

| Rule | Port | Source               | Purpose                         |
|------|------|----------------------|---------------------------------|
| HTTP | 80   | 0.0.0.0/0            | View website                    |
| SSH  | 22   | Temporary lab access | GitHub-hosted runner connection |

**For this classroom implementation, SSH was temporarily opened to 0.0.0.0/0 because the GitHub-hosted runner does not have the trainer's fixed IP. This is a lab workaround; tighten it after the demo.**

#### Connect and install Nginx

```
ssh -i <your-key.pem> ubuntu@<EC2-PUBLIC-IP>
```

```
sudo apt update
sudo apt install nginx -y
sudo systemctl enable nginx
sudo systemctl start nginx
sudo systemctl status nginx
```

Open <http://> and verify the default Nginx page.

## 4. Create the demo application

Create a folder named devops-github-demo and add these files.

### index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>DevOps CI/CD Demo</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1>DevOps CI/CD Demo</h1>
    <p>This application is deployed using
      <strong>GitHub Actions</strong> to an
      <strong>AWS EC2</strong> instance.</p>
    <div class="info">
      <p><strong>Source:</strong> GitHub</p>
      <p><strong>CI/CD:</strong> GitHub Actions</p>
      <p><strong>Server:</strong> AWS EC2</p>
      <p><strong>Web Server:</strong> Nginx</p>
    </div>
  </div>
</body>
</html>
```

### style.css

```
body { font-family: Arial, sans-serif; margin: 0; background: #f4f6f8; }
.container { width: 70%; margin: 100px auto; padding: 40px; background: white;
  text-align: center; border-radius: 10px; box-shadow: 0 4px 15px rgba(0,0,0,0.1); }
h1 { margin-bottom: 20px; }
.info { margin-top: 30px; padding: 20px; background: #f0f3f6; border-radius: 8px; }
```

## 5. Initialize Git and create the GitHub repository

```
git --version
git init
git status
git add .
git commit -m "Initial application"
git branch -M main
```

Create a new GitHub repository named **devops-github-demo**. For a clean lab, do not initialize it with another README, .gitignore or license.

```
git remote add origin <YOUR-GITHUB-REPOSITORY-URL>
git remote -v
git push -u origin main
```

**Verify that index.html and style.css are visible in the GitHub repository.**

## 6. Configure GitHub Secrets

GitHub Actions runs on a separate runner, so it needs secure credentials to reach EC2. Never commit the EC2 private key to Git.

**On EC2, make the Nginx directory writable by the ubuntu deployment user:**

```
sudo chown -R ubuntu:ubuntu /var/www/html  
ls -ld /var/www/html
```

In GitHub: Repository → Settings → Secrets and variables → Actions → New repository secret.

| Secret      | Value                                         |
|-------------|-----------------------------------------------|
| EC2_SSH_KEY | Complete contents of the EC2 .pem private key |
| EC2_HOST    | EC2 public IPv4 address only                  |
| EC2_USER    | ubuntu                                        |

**Security rule: do not paste the private key into YAML, README files, source code or screenshots.**

## 7. Create the GitHub Actions workflow

Create: `.github/workflows/deploy.yml`

name: Deploy to AWS EC2

on:

push:  
 branches:  
 - main

jobs:

deploy:  
 runs-on: ubuntu-latest

steps:

- name: Checkout source code  
 uses: actions/checkout@v4

- name: Test application  
 run: |  
 test -f index.html  
 test -f style.css  
 echo "Application files found successfully"

- name: Configure SSH  
 run: |  
 mkdir -p ~/.ssh  
 echo "\${{ secrets.EC2\_SSH\_KEY }}" > ~/.ssh/ec2\_key  
 chmod 600 ~/.ssh/ec2\_key  
 ssh-keyscan -H "\${{ secrets.EC2\_HOST }}" >> ~/.ssh/known\_hosts

- name: Deploy application  
 run: |  
 scp -i ~/.ssh/ec2\_key index.html style.css "\${{ secrets.EC2\_USER }}"@\${{

- name: Verify deployment  
 run: |  
 ssh -i ~/.ssh/ec2\_key "\${{ secrets.EC2\_USER }}"@\${{ secrets.EC2\_HOST }} "ls -l /var/w

### What each part does

| Part             | Purpose                                         |
|------------------|-------------------------------------------------|
| on → push → main | Starts the workflow after a push to main.       |
| jobs → deploy    | Defines the deployment job.                     |
| runs-on          | Uses a GitHub-hosted Ubuntu runner.             |
| checkout         | Downloads repository code onto the runner.      |
| Test             | Checks that index.html and style.css exist.     |
| Configure SSH    | Creates the key file and verifies the EC2 host. |
| scp              | Copies files to /var/www/html.                  |
| ssh verify       | Connects to EC2 and lists the deployed files.   |

## 8. Commit, push and run

```
git add .  
git commit -m "Add GitHub Actions deployment workflow"  
git push
```

Open GitHub → Actions → Deploy to AWS EC2. The expected flow is:

```
Set up job  
↓  
Checkout source code  
↓  
Test application  
↓  
Configure SSH  
↓  
Deploy application  
↓  
Verify deployment  
↓  
Complete job
```

**A successful run means the runner checked out and validated the code, connected to EC2, copied the files and verified them.**

Then open <http://>. The custom DevOps CI/CD page should replace the default Nginx page.

## 9. Successful workflow example

← Deploy to AWS EC2

✓ Add GitHub Actions deployment workflow #2

Re-run all jobs Latest #2 ⋮

Summary

All jobs

✓ deploy

Run details

Usage

Workflow file

> Annotations

1 warning and 1 notice

deploy

succeeded now in 6s

Search logs

↺ ⚙

> ✓ Set up job

> ✓ Checkout source code

> ✓ Test application

> ✓ Configure SSH

> ✓ Deploy application

> ✓ Verify deployment

> ✓ Post Checkout source code

> ✓ Complete job

1s

0s

0s

1s

1s

1s

0s

0s

This run shows Checkout, Test, Configure SSH, Deploy and Verify all succeeding.

# 10. Troubleshooting

## A. “No event triggers defined in on”

Ensure the workflow trigger is exactly:

```
on:  
  push:  
    branches:  
      - main
```

Use spaces, not tabs, and make sure the branch is actually named main.

## B. Configure SSH fails at ssh-keyscan

If the failure occurs at ssh-keyscan, the runner cannot reach EC2 on TCP 22. In this lab, the cause was an EC2 security group that allowed SSH only from the trainer's IP. A GitHub-hosted runner comes from a different public IP.

**For the short classroom lab, allow TCP 22 from 0.0.0.0/0 temporarily, rerun the failed job, and remove/restrict the rule afterward.**

This is a network/security issue, not a GitHub Actions YAML issue.

# 11. Demonstrate continuous deployment

Change the application heading:

```
<h1>DevOps CI/CD Demo</h1>
```

to:

```
<h1>Version 2 - Automatically Deployed!</h1>
```

Then push the change:

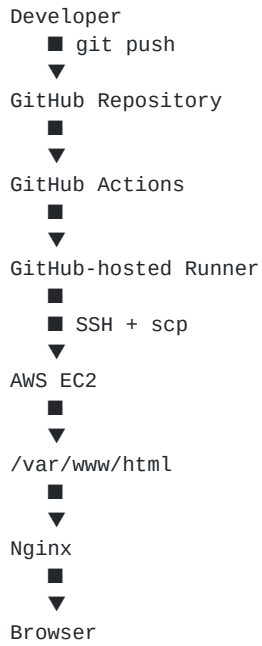
```
git add .  
git commit -m "Update application to version 2"  
git push
```

**The push automatically starts the workflow again. After it succeeds, refresh the EC2 URL. The new heading should appear.**

Teaching message: “I changed the source code, committed it and pushed it. I did not manually log in to the server to deploy the change. The pipeline performed the deployment.”

This demonstrates the practical value of continuous deployment.

## 12. Final architecture and concepts



| Concept        | What students should remember                    |
|----------------|--------------------------------------------------|
| Trigger        | A push to main starts the workflow.              |
| CI             | Automated checkout and validation.               |
| CD             | Automated deployment to EC2.                     |
| Runner         | Executes the workflow; it is not the EC2 server. |
| Secrets        | Sensitive values stay outside source code.       |
| Security group | Controls whether the runner can reach EC2.       |
| Nginx          | Serves the files deployed to /var/www/html.      |

## 13. Production considerations

The classroom implementation is intentionally simple. For production, review:

- Do not permanently expose SSH TCP 22 to 0.0.0.0/0.
- Use least-privilege access and short-lived credentials where practical.
- Consider AWS Systems Manager and IAM-based access to reduce dependence on public SSH.
- Use stronger deployment controls and environment separation for production.
- Use HTTPS and an appropriate front-end/load-balancing architecture for real applications.
- Use proper automated tests rather than only file-existence checks.
- Review and maintain third-party GitHub Actions used by workflows.

**After the classroom demo, remove the temporary public SSH rule or restore a restricted source range.**

## 14. Cleanup checklist

- Stop or terminate the EC2 instance when it is no longer needed.
- Restrict/remove public SSH access after the lab.
- Remove unused security groups and other resources where appropriate.
- Never leave private keys in the repository.
- Check AWS billing for actual account-specific costs.

## 15. Student recreation checklist

| Done | Task                              |
|------|-----------------------------------|
| ■    | Launch Ubuntu EC2                 |
| ■    | Allow HTTP 80                     |
| ■    | Prepare SSH access for the lab    |
| ■    | Install and verify Nginx          |
| ■    | Create HTML/CSS application       |
| ■    | Initialize Git and commit         |
| ■    | Create GitHub repository          |
| ■    | Push source to GitHub             |
| ■    | Create EC2_SSH_KEY secret         |
| ■    | Create EC2_HOST secret            |
| ■    | Create EC2_USER secret            |
| ■    | Create deploy.yml                 |
| ■    | Push workflow                     |
| ■    | Verify successful Actions run     |
| ■    | Verify website on EC2             |
| ■    | Change application and push again |
| ■    | Observe automatic redeployment    |
| ■    | Clean up/restrict SSH             |

**End result: students can recreate the same GitHub Actions → AWS EC2 CI/CD pipeline in their own accounts.**